Geometry Is Not a Prompt: What Physical-World Agents Should Compute Instead of Infer

Joan Vendrell Gallart
KavAI & UCI
San Francisco, CA 94109
jvendrel@uci.edu

Elaheh Vaezpour
KavAI
San Francisco, CA 94109
elaheh.vaezpour@kavai.com

Behnam Moradi
KavAI
San Francisco, CA 94109
behnam.moradi@kavai.com

Tara Javidi
KavAI & UCSD
San Francisco, CA 94109
tara.javidi@kavai.com

Abstract

Physical-world agents often combine two different kinds of reasoning: interpreting flexible language and computing quantities determined by the physical world. Treating both as prompting problems can be unreliable, particularly when correctness depends on camera pose, field of view or geometric projection. We formulate *mechanism allocation*: a task is divided into sequential sub-tasks, and each is assigned either to an LLM or to a specialized tool based on its expected error, latency, cost, and flexibility. We use these quantities to compare complete agent designs and study how errors in their estimation affect the selected allocation. We apply the framework to asset-aware retrieval of images in real industrial applications, where an agent aims to retrieve images containing a given asset under distinct requirements. The study compares direct prompting, prompts augmented with equations, model-generated code, and deterministic geometric tools. The aim is not to replace the LLM, but to retain it for decisions requiring language understanding while computing explicitly what is best computed rather than inferred.

1 Introduction

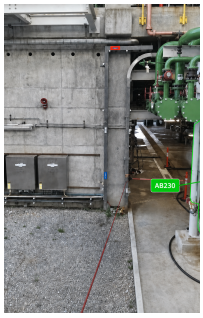
Language models are increasingly embedded in systems that observe and act on the physical world: inspecting infrastructure, retrieving views of an object, or navigating a site. Their appeal is flexibility; they interpret natural-language objectives, combine heterogeneous information, and adapt to tasks nobody programmed. But physical action depends on precise facts about location, orientation, visibility and geometry, so spatial competence is a gating requirement: *a system that cannot reliably determine where things are cannot be trusted to act around them.*

Language models remain poorly suited to that requirement. They lack access to the geometric data many physical questions depend on (Yu et al., 2025); multimodal models miss spatial inconsistencies obvious to humans (Khangaonkar et al., 2026); performance on Earth imagery stays well below human ability on tasks reducible to metric reasoning (Xu et al., 2026); and spatial competence decomposes into sub-skills that improve unevenly with scale (Xiao et al., 2026). This is unsurprising: models operate over tokenized context and produce outputs autoregressively, whereas these questions are settled by continuous measurements, coordinate transformations and camera models. Throughout, *language model* is an umbrella term; e.g. *Gemini* operates as a multimodal language model (MLLM) when given images and as a text-based LLM when given language and structured metadata.

NeurIPS submission.



(a) Tag legible: correct box.



(b) LLM confident, but wrong.



(c) FOV tool output.

Figure 1: Single-call visual grounding with `gemini-3.7-flash`, prompted "*Highlight the vessel AB230.*" (a) When the stenciled tag is legible, the returned box tightly and correctly localizes the asset. (b) In an oblique view where the same asset is cropped at the right border and the tag is unreadable, the model returns `found=true` but misidentifies it (ground truth in green). (c) A deterministic FOV computation avoids this semantic inference, but certifies only geometric inclusion and does not address the remaining grounding requirements.

Agentic AI addresses the mismatch by equipping the model with tools (Schick et al., 2023; Gao et al., 2023), retaining linguistic flexibility while delegating geometric or numerical operations to mechanisms designed for them. But providing tools does not say how responsibilities should be divided among them. That division is a design-time problem—which sub-tasks stay with the LLM and which are assigned elsewhere—and existing work studies *tool invocation* at inference time, leaving *mechanism allocation* implicit. Assigning too much to the model compromises reliability; delegating too much adds overhead and removes flexibility. Hence, a criterion that allocates sub-tasks by jointly comparing accuracy, latency, cost and flexibility of inference against computation is missing.

For example, consider an operator at an industrial site asking for images of a particular asset, perhaps from a given orientation, at close range, or from distinct viewpoints. The request seems simple to a human, yet Fig. 1 shows that an LLM may identify the wrong element. Such errors matter: an incorrect asset or an image in which the asset is not visible can mislead the operation that follows. A fixed tool is not enough either especially when the request has semantic information such as *show me a thermal image*, *show the asset from the right*, or *find a view that also contains asset AB252*, which would imply to implement a specific tool for every possible operation. The design problem is therefore to decide which requirements need precise computation and which should remain with the LLM to preserve this flexibility.

To address this challenge, we formulate mechanism allocation as a measurable, stepwise design problem. Given a functional requirement, a designer—human or AI—first decomposes the objective into sub-tasks. Each candidate mechanism then declares its inputs, capabilities, limitations, and estimated error, latency, cost, and flexibility. These descriptions define a common assignment measure used to decide whether each sub-task should remain with the LLM or be delegated to a deterministic tool, as illustrated in Fig. 2. Since allocation depends on mechanism descriptions, tools can be added or replaced without redesigning the complete agent. We study this framework on real industrial tasks.

Contributions.

1. A decision-theoretic formulation of per-step allocation between LLM inference and tool computation, with a separable upper bound on task risk that requires no independence assumption and a regret bound for inaccurate mechanism descriptions (§4).
2. A four-part study of how allocation is carried out in practice: discerning sub-tasks from a functional requirement, identifying which steps depend on semantic context, testing whether geometric knowledge can be conveyed through prompting instead of computation, and examining when an LLM-synthesized procedure can become a reliable reusable tool (§5).
3. An empirical evaluation on asset-aware retrieval at a real industrial site, using 109 catalogued assets and $\sim$31k geo-located frames with camera pose. The evaluation includes 24 previously unused assets, ground truth from an independent projection oracle and pixel adjudication, and a comparison of the allocation rule against all 16 admissible allocations (§5.5).

2 Related Work

Tool use and tool invocation. Toolformer (Schick et al., 2023) and PAL (Gao et al., 2023) show that delegating computation to external calls improves reliability, and both study *when to invoke* a tool at inference time. We study the prior question: given a decomposition, which sub-tasks should be assigned to a tool at design time, and on what measured grounds.

Geospatial retrieval. Retrieval-augmented generation grounds outputs in retrieved evidence (Lewis et al., 2020); `Spatial-RAG` extends this to geographic queries (Yu et al., 2025) and `CivicEmbed` learns embeddings for geographic retrieval (Wang et al., 2026a). These return entities geographically or semantically related to a query. Our task needs a stricter relation, *visual grounding*: whether an asset appears in an image depends on camera position, orientation and field of view, which no distance- or embedding-based retriever recovers. Diverse subset selection (Kulesza and Taskar, 2012; Carbonell and Goldstein, 1998) addresses the complementary redundancy problem; we apply farthest-point selection only after a visibility filter, and Appendix I shows the order matters more than the selector.

Physical geometry and explicit computation. Benchmarks report that multimodal models fail on metric spatial reasoning (Xu et al., 2026; Khangaonkar et al., 2026; Xiao et al., 2026). We take the complementary position: on this task the required computation is a projection with known parameters, so the design question is which mechanism performs it, not whether a model can be trained to.

3 Problem Statement

Let a task be a map $h : \mathcal{X} \to \mathcal{Y}$, where $x \in \mathcal{X}$ is an input and $h(x) \in \mathcal{Y}$ is the desired output. Let us assume that h can be sequentially decomposed such as

$$h = h_K \circ \cdots \circ h_1, \tag{1}$$

where $h_k : \mathcal{Z}_{k-1} \to \mathcal{Z}_k$ is the k-th sub-task, with $\mathcal{Z}_0 = \mathcal{X}$ and $\mathcal{Z}_K = \mathcal{Y}$. Then, for each sub-task, the designer may use either LLM inference f_k or a specialized tool τ_k. Hence, an allocation $a = (a_1, \ldots, a_K)$, where $a_k \in \mathcal{M}_k := \{f_k, \tau_k\}$, defines the agentic system inference pipeline for the given task h as

$$\widehat{h}_a := a_K \circ \cdots \circ a_1. \tag{2}$$

In order to quantify allocation quality, in this work we propose to measure each mechanism $m \in \mathcal{M}_k$ with four quantitative measures: the error probability $e_k(m) \in [0, 1]$, the expected execution time $t_k(m)$, the expected computational cost $c_k(m)$ and the flexibility $\phi_k(m) \in [0, 1]$ computed as the fraction of variants of h_k that m can execute without retraining, reprogramming, or changing its interface. These quantities are calibrated before deployment from mechanism specifications, calibration data, or empirical evaluation. The input distribution and correctness contract determine e_k, the hardware, API and retrieval architecture determine t_k and c_k, while the request-variant distribution determines ϕ_k. The reported fraction of supported variants uses uniform weights on the stated test suite; for deployment frequencies $p(v)$ it becomes $\phi_k(m) = \sum_v p(v)\mathbf{1}_{\{m \text{ supports } v_k\}}$, where v_k is the sub-task variant induced by request v. The same distribution over complete requests must underlie all sub-task flexibility estimates.

Define each sub-task error relative to the correct intermediate state in the decomposition (1). The complete task can then fail only if at least one such sub-task fails, and the union bound gives

$$\Pr\left(\widehat{h}_a(x) \neq h(x)\right) \leq \overline{E}(a) := \sum_{k=1}^{K} e_k(a_k), \tag{3}$$

without requiring independence between sub-task failures. Under sequential execution and additive resource accounting that includes local overhead, the corresponding time and cost satisfy

$$T(a) \leq \overline{T}(a) := \sum_{k=1}^{K} t_k(a_k), \qquad C(a) \leq \overline{C}(a) := \sum_{k=1}^{K} c_k(a_k), \tag{4}$$

while, similarly, since the system can accommodate a task variant only if every selected mechanism supports its corresponding sub-task, its flexibility admits the lower bound

$$\Phi(a) \geq \underline{\Phi}(a) := 1 - \sum_{k=1}^{K} \big(1 - \phi_k(a_k)\big). \tag{5}$$

Let $E(a) := \Pr(\widehat{h}_a(x) \neq h(x))$. We evaluate an agent design through the weighted success score

$$S(a) := w_{\mathrm{A}}\,(1 - E(a)) - w_{\mathrm{T}}\frac{T(a)}{T_0} - w_{\mathrm{C}}\frac{C(a)}{C_0} + w_{\mathrm{F}}\Phi(a), \tag{6}$$

where $T_0, C_0 > 0$ are normalization constants and $w_{\mathrm{A}}, w_{\mathrm{T}}, w_{\mathrm{C}}, w_{\mathrm{F}} \geq 0$ encode the design priorities. Equations (3)–(5) give the computable lower certificate

$$\underline{S}(a) := w_{\mathrm{A}}\big(1 - \overline{E}(a)\big) - w_{\mathrm{T}}\frac{\overline{T}(a)}{T_0} - w_{\mathrm{C}}\frac{\overline{C}(a)}{C_0} + w_{\mathrm{F}}\underline{\Phi}(a) \leq S(a). \tag{7}$$

The agent-design problem is therefore the allocation

$$a^\star \in \arg\max_{a \in \mathcal{M}_1 \times \cdots \times \mathcal{M}_K} \underline{S}(a). \tag{8}$$

Thus, the design reduces to assigning each sub-task to the LLM or its specialized tool according to their measured reliability, latency, cost, and flexibility.

4 Method: Success-Guided Agent Design

Section 3 assumes the decomposition and the candidate mechanisms are known; in practice neither is. We therefore give a design-time procedure starting from a task specification ρ and a calibration set $\mathcal{B}$: it produces a decomposition, evaluates an all-LLM baseline, builds tools for poorly served sub-tasks, and retains one only when it improves the certificate in (7).

A planner maps ρ into a plan $(\pi, \Sigma) = \mathrm{Plan}(\rho)$, where $\pi = (h_1, \ldots, h_K)$ is the decomposition of (1) and σ_k is sub-task h_k's contract, declaring its input and output schema (Figure 2). The initial mechanism set holds no tools, $\mathcal{M}_k^{(0)} = \{f_k\}$, and this baseline is evaluated on $\mathcal{B}$ for error, latency, cost and flexibility at every sub-task.

Second, for a mechanism m assigned to sub-task k, define its estimated local design loss as

$$\widehat{d}_k(m) := w_{\mathrm{A}}\widehat{e}_k(m) + w_{\mathrm{T}}\frac{\widehat{t}_k(m)}{T_0} + w_{\mathrm{C}}\frac{\widehat{c}_k(m)}{C_0} + w_{\mathrm{F}}\big(1 - \widehat{\phi}_k(m)\big), \tag{9}$$

which estimates are obtained from contract-based tests, repeated executions, resource profiling, and evaluation over admissible task variants. The evaluator uses it to identify the sub-tasks for which the current mechanism provides the weakest accuracy–latency–cost–flexibility trade-off.

Using (9) we then build mechanisms for the weakest sub-tasks. For a selected k, a tool maker receives σ_k, calibration examples and observed failures, and returns either a candidate tool

$$\tau_k = \mathrm{MakeTool}(\sigma_k, \mathcal{B}_k)$$

or $\varnothing$ when no suitable explicit mechanism can be constructed. As with planning, tool construction may be performed by an LLM, a human, or an LLM with human review. A tool may implement, for example, a numerical routine, geometric computation, database query, program, or external API call.

To be admissible the candidate must satisfy σ_k's interface, pass its correctness tests, and declare unsupported inputs and known limitations. The evaluator measures $\widehat{d}_k(\tau_k)$ on the protocol used for f_k, and replacing a_k by τ_k gives the marginal gain

$$\widehat{\Delta}_k(\tau_k) := \widehat{\underline{S}}(a_{-k}, \tau_k) - \widehat{\underline{S}}(a) = \widehat{d}_k(a_k) - \widehat{d}_k(\tau_k).$$

Hence, the tool is retained only if $\widehat{\Delta}_k(\tau_k) > \delta_k$, where $\delta_k \geq 0$ is a calibration margin. Otherwise, the current mechanism is preserved. The process stops after all selected sub-tasks are examined, no candidate produces sufficient gain, or the design budget is exhausted.

The guarantee is relative to the mechanisms actually generated. Once the process terminates the composition is frozen and executed directly on new instances.

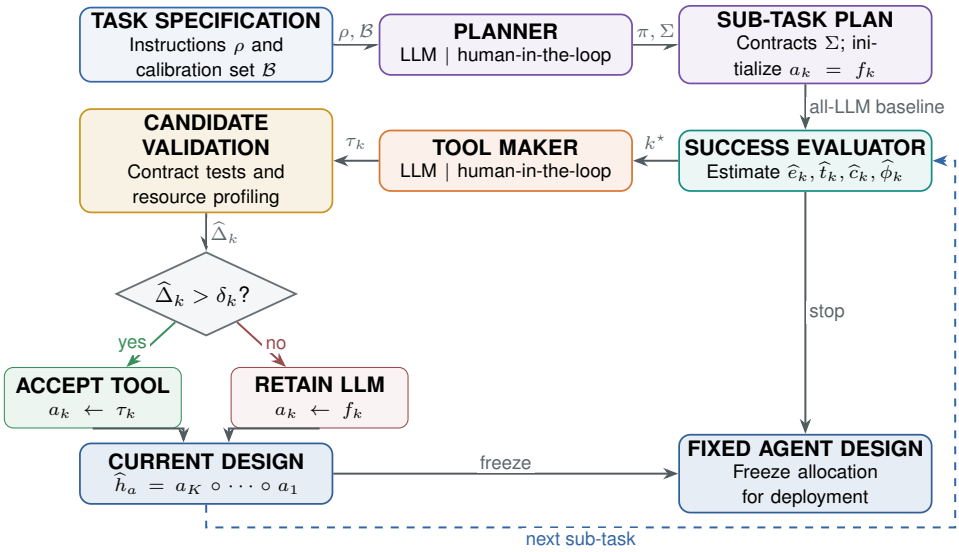


Figure 2: Success-guided agent design. Solid arrows denote design signals, while the dashed arrow denotes iterative refinement. Planning and tool construction may be autonomous or human-assisted, and the final allocation is frozen before deployment.

Proposition 1 (Regret under estimated mechanism attributes)**.** *Let $d_k(m)$ denote the true counterpart of $\widehat{d}_k(m)$, and suppose $\left|\widehat{d}_k(m) - d_k(m)\right| \le \eta_k$ for every $m \in \mathcal{M}_k$. Let $\widehat{a}_k \in \arg\min_{m\in\mathcal{M}_k} \widehat{d}_k(m)$ and $a_k^\star \in \arg\min_{m\in\mathcal{M}_k} d_k(m)$. Then*

$$\underline{S}(a^\star) - \underline{S}(\widehat{a}) \le 2\sum_{k=1}^{K} \eta_k. \tag{10}$$

Moreover, choosing $\delta_k \ge 2\eta_k$ ensures that every accepted tool improves the true local design loss.

The proof is a three-line telescoping argument and is given in Appendix A.

5 Case Study: Asset-Aware Retrieval

We instantiate the design procedure of Section 4 on a task family drawn from industrial asset inspection. A site is surveyed by drone and by hand; every frame carries a camera pose — position, compass yaw, gimbal pitch, altitude and, where recorded, a 35 mm-equivalent focal length — and every asset carries a tag, a geodetic position and, when surveyed, a ground extent and height. The site used here holds 109 catalogued assets and ∼31k geo-located frames, of which ∼15k lie within 1 km of the asset set; Appendix B reports the spatial distribution and the metadata gaps.

Four cases are studied. **Case A** asks *where* a named asset appears inside a frame; **Case B** asks *which* frames show it; **Case C** adds a request-time selection constraint; and **Case D** delegates construction of the tool itself. In Cases A and B, the answer is fixed by geometric projection rather than lexical or semantic similarity, but inference fails for different reasons: Case A withholds the catalogue correspondence needed for identity, whereas Case B supplies the pose variables but requires their explicit composition. Throughout we use $w_{\mathrm{A}} = 1$, $w_{\mathrm{T}} = 0.3$, $w_{\mathrm{C}} = 0.2$, $w_{\mathrm{F}} = 0.2$, $T_0 = 1\,\mathrm{s}$ and $C_0 = 10^4$ tokens; prompts, model configurations, and the estimation protocol appear in Appendices D and E.

5.1 Case A: visual grounding in a single frame

The given functionality is to provide the agent with the capacity to identify where an asset is in a frame. The planner decomposes the task into resolving the tag to a catalogue entry and extent (h_1), localizing the asset in the image (h_2), and returning the region (h_3); the all-LLM baseline merges h_1 and h_2 into one multimodal call. Figure 1 shows it succeeding when `AB230` is readable

and failing confidently when the same asset is viewed obliquely with the tag cropped. Visually similar assets are distinguished only by catalogue information, so image-only reasoning cannot recover the correspondence. Applying the proposed framework, h_2 is replaced with a projection tool (Appendix C) costing $\sim$0.02 ms per frame. Under the stated pinhole-camera, ground-plane, and metadata assumptions, it returns a projected region or $\varnothing$ rather than forcing a plausible box. Then, the model is retained for appearance-dependent properties h_4 such as occlusion and blur.

5.2 Case B: asset-aware image retrieval

This case studies a broader functionality, to be able to select, from a pool of images, best κ images that contains certain asset. As it can be seen, it is one step over Case A, and requires from much more efficient pipeline than just observing image by image. The planner decomposes the request into four steps: parse the target ρ and count κ (h_1), retrieve candidate frames and pose metadata (h_2), decide whether each contains the asset (h_3), and select κ distinct viewpoints from the valid ones (h_4). Separating visibility from diversity matters because diversity is meaningful only after invalid frames are removed. On the development query that motivated the design (Appendix E) every all-LLM configuration returns 0% in-frame; held out, the same prompt reaches $0.614\,[0.476,\ 0.750]$ (§5.5), so that query bounds a failure mode rather than measuring it. It is relevant to highlight that supplying the exact projection equations does not repair it ($0.653\,[0.521,\ 0.781]$), which supports our motivating statement that it is not enough on providing the geometry to the prompt.

The tool maker supplies three mechanisms. First, τ_2 retrieves $1{,}000$ nearby frames in one bulk query (0.89 s), replacing model-controlled pagination that produced a proximity-biased pool with no valid frames. Second, τ_3 is a projection operator that projects the image corners onto the ground plane and tests whether the footprint contains the asset (0.0215 ms per frame), returning *pose unknown* when the camera cannot be oriented. And, finally, τ_4 applies farthest-point selection to in 0.2 ms.

The mechanism allocation shows that the parser τ_1 is faster than f_1 but far less flexible across paraphrases, so the criterion retains f_1 and delegates only h_2–h_4; Appendix E gives the arithmetic ($\widehat{\Delta}_2 = +1.63$, $\widehat{\Delta}_3 = +7.63$, $\widehat{\Delta}_4 = +0.73$ against $\widehat{\Delta}_1 = -0.14$). The gain depends on decomposition as much as on mechanism quality—farthest-point selection reaches 50% in-frame accuracy on the unfiltered pool and 100% after τ_3 filters it, the same mechanism in a different order—and the cost in flexibility is explicit, with $w_{\mathrm{F}} \gtrsim 1.4$ first reversing h_4.

5.3 Case C: constrained retrieval, and when a criterion can be compiled

Meanwhile Case B fixes the selection objective: "the κ clearest views" always means spread the viewpoints out, so σ_4 reads *select κ diverse viewpoints* and a tool implements it once. Case C asks "κ views of asset ρ that c", where the criterion c arrives with the request—overhead, from the north side, thermal, close-up colour for a corrosion check, not overhead and not thermal, and six others (Appendix F). The first three steps keep Case B's allocation; only h_4 is reopened, over the identical in-FOV pool, and σ_4 now reads *select κ candidates satisfying c.*

To open the required functionality to broader spectra, shows the restrictions on the previous final decision tools as neither `farthest_point_selection(xyz, quality, k)` nor `top_k_nearest(problem, k)` has anywhere to put c. Scored anyway, the diversity selector reaches mean precision 0.327 and clears the 0.8 service threshold on one criterion of eleven—its own—against 0.718 and seven for f_4 (Appendix Figure 5(a)).

What h_4 requires is a mechanism that can *receive* c; whether that mechanism is a model in the serving path or a program compiled from c beforehand depends on one thing, whether the criterion space is closed. Extending this experiment, we introduced τ_4^{synth}, where the model is asked *once, offline*, to write a Python scoring function for each criterion, which is then executed deterministically at serving time. It reaches mean precision 0.818 and serves 9 of 11 criteria, against 0.718 and 7 for the model, at 0.025 s per request instead of 2.60 s and no serving tokens at all—design loss 0.226 against 1.226. However, compilation fails here exactly where the request is ambiguous rather than where it is complex as τ_4^{synth} miss is the *spread*, while the model admits more than one threshold, not the compound constraints. Hence, the model is retained for the open tail, and the design question becomes how large that tail is, which is a property of the deployment, not of the mechanism.

5.4 Case D: the tool maker is an LLM

Similarly to τ_4^{synth}, in Case D we study an autonomous $\mathrm{MakeTool}$ on demand: given a sub-task h_i a model receives three worked examples and a prose description of the acceptance suite, and returns a Python tool, never seeing the reference implementation or the test code (Appendix G). Specifically, the model receives 13 synthetic tests with 8 *core* tests that cover the basic geometry: cameras facing toward, away from, and sideways to the target, distant and overhead nadir views, and a frame with no camera position; and 5 *field* tests which encode how pose metadata must be interpreted in this corpus: missing yaw, missing pitch, zero relative altitude used as a missing value, and a shallow-pitch view whose projected footprint must be limited before reaching the horizon.

We then vary only the specification given to the model. The *full* arm receives the 5 *field* conditions and must pass all 13 tests, whereas the *core* arm is not told about them and is tested only on the eight generic cases. We run 10 independent synthesis attempts per arm. After each attempt, the generated code is executed; if it fails or does not compile, the model receives the failing assertions and may repair the code up to 3 times. We report pass@n, the fraction of attempts that produce a tool passing every test in their gate within n versions (Chen et al., 2021).

However, as passing synthetic tests does not show that the tool transfers to real data, we also run every accepted tool on 450 held-out (frame, target) pairs from the frozen assets of §5.5, balanced across verdict classes. Each prediction is compared with 3 references serving different purposes. First, the deployed tool measures whether the synthesized tool reproduces the current system; second, the independent projection oracle measures agreement with a separate geometric implementation; and, third, for 310 pairs, the pixel judge determines visibility directly from the images. Agreement with the deployed tool is not treated as ground truth, since it would reward a synthesized tool for reproducing the incumbent's errors. We count abstentions as incorrect, otherwise a tool could improve its apparent score simply by refusing difficult frames. Under this evaluation, *full*/*core* score 0.834/0.650 against the deployed tool, 0.716/0.669 against the oracle, and 0.537/0.342 against the pixel labels.

The experiment supports a narrower conclusion than saying that an LLM can autonomously create reliable tools. It can translate a stated contract into code and repair that code until it passes a fixed suite, but it cannot recover domain assumptions that are absent from the specification. Indeed, the specification and its independent validation, rather than the identity of the tool maker, determine whether the resulting tool is reliable.

5.5 Frozen held-out evaluation

Cases A–D are development studies: their targets were used while the method was being designed, and their visibility labels were produced by the same geometric model used by the deployed tool. We remove both dependencies by freezing 24 assets disjoint from every development target, selected by a rule fixed in advance (Appendix H), and retrieving their nearest 400 frames. We then conduct two evaluations. First, the independent oracle labels all 9600 frames to assess the deployed visibility tool over the complete pool. Second, we compare six end-to-end configurations: the deployed LLM prompt, the LLM given the projection equations, the LLM writing and executing code, the LLM with the FOV tool, the deterministic pipeline, and our selected allocation, see Table 2. For this comparison, every configuration receives the same nearest 120 frames per asset. The larger pool does not fit within the prompted baselines' context, and using different pool sizes would confound the effect of the mechanism with the number of candidates examined.

Oracle Definition. To evaluate the frozen benchmark, we first need visibility labels that do not inherit the assumptions of the deployed system. We therefore construct an independent oracle that projects the target into the image plane using the recorded focal length, gimbal roll, and full 3-D rotation, and checks whether the projection falls inside the sensor rectangle. This differs from the deployed FOV tool τ_3 from Case B, which projects the four camera corners onto a ground plane and tests whether the resulting footprint contains the target. The two implementations agree on 0.924 $[0.918,\ 0.931]$ of the 6619 frames for which both give a verdict. This comparison shows how often the two geometric methods coincide, but not which one is correct when they disagree.

We therefore adjudicate 435 frames directly from the images. The judge sees only a reference photograph and the candidate frame, without pose data, coordinates, or either geometric verdict. A human also labels all disputed frames and a sample of the agreeing cases. The independent

Table 1: Visibility precision, recall, and F1 against pixel labels. The merged reference uses human labels where available and automatic adjudication otherwise, with sampling-design reweighting (Appendix H); the human-only subset is reported separately. F1 is the harmonic mean of the reported, rounded precision and recall estimates, not an average of per-target F1 scores.

Reference	Mechanism	Precision	Recall	F1
Merged pixel labels	Deployed FOV tool	0.900	0.131	0.229
	Independent oracle	0.833	0.687	0.753
Human-only subset	Deployed FOV tool	1.000	0.110	0.198
	Independent oracle	0.837	0.932	0.882

Table 2: Frozen held-out evaluation: 24 assets disjoint from every development target, 120 frames each, $\kappa = 10$, 3 repeats per target, scored against the independent oracle. **Precision** is strict—a returned frame the oracle will not rule on counts against the arm. **Coverage** is the fraction of the $\min(\kappa, \text{available})$ reachable frames found. Brackets are 95% bootstrap intervals resampling *targets*. *Gemini 3.7* shown; the second model reproduces the ordering (Appendix H).

Mechanism at h_3, h_4	Precision (strict)	Coverage@κ	Latency	Tokens
LLM, deployed prompt	0.614 [0.476, 0.750]	0.796 [0.697, 0.893]	6.33 s	6893
LLM, exact equations given	0.653 [0.521, 0.781]	0.768 [0.669, 0.864]	8.19 s	7640
LLM writes and runs code	0.982 [0.951, 1.000]	0.865 [0.803, 0.925]	5.70 s	1747
LLM + FOV tool	0.973 [0.927, 1.000]	0.753 [0.610, 0.887]	3.87 s	926
Deterministic pipeline	0.982 [0.946, 1.000]	0.757 [0.615, 0.891]	0.01 s	0
Our allocation	0.982 [0.946, 1.000]	0.757 [0.615, 0.891]	0.83 s	101

implementation and the pixel adjudication together support using the oracle as the reference for the frozen evaluation. They also correct our original estimate of τ_3: the tool is precise when it accepts a frame, but it rejects many frames in which the asset is visible. Case B did not reveal this because $\widehat{e}_3(\tau_3)$ was estimated using τ_3's own model of visibility, but the practical conclusion remains.

Precision, recall, and F1. For binary visibility evaluation, $P = \mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$, $R = \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$, and $F_1 = 2PR/(P + R)$. Table 1 reports all three against the pixel reference. The deployed tool's high precision comes with low recall and F1, so geometric filtering does not establish exhaustive retrieval. The oracle has a more balanced operating point, while still being an imperfect reference. The human-only subset gives the same qualitative trade-off.

Methods comparison. Then, every arm answers the same request over the same pool. Table 2 revises the paper's claims in two directions. Supplying the equations does not repair the failure as the model has the formulas and still composes them unreliably, while letting it *write* and *run* the computation repairs the precision entirely ($0.982\,[0.951,\ 1.000]$). The precision gain persists across the two evaluated models, but completeness varies substantially; coverage is $0.865\,[0.803,\ 0.925]$ on *Gemini 3.7* and $0.476\,[0.344,\ 0.605]$ on *Gemini 2.5*, against $0.757\,[0.615,\ 0.891]$ for the pipeline computed as

$$\text{Coverage}\,@\kappa(q) = \frac{|R_\kappa(q) \cap V(q)|}{\min\{\kappa, |V(q)|\}}, \tag{11}$$

where $R_\kappa(q)$ is the set of frames returned for query q, and $V(q)$ is the set of available frames in which the target is visible. The accepted allocation matches the best precision at 0.83 s and 101 tokens—17× fewer than code generation, 68× fewer than prompting—with no generated code in the serving path, and its ordering is identical on both models.

Coverage@κ (11) measures how much of the requested quota is filled with valid frames, not its standard recall. Indeed, coverage is equal to the recall only if $|V(q)| \leq \kappa$. Per-query recall and F1 for the six retrieval arms cannot be reconstructed from the aggregate precision and coverage in Table 2; this requires the returned identifiers and positive counts for each query. We therefore report the available binary-visibility recall and F1 separately in Table 1, without treating them as end-to-end retrieval scores.

Allocation Mechanism. Finally, we test whether the local allocation rule recovers a good complete design. Following Case B, with 4 binary sub-tasks, there are 16 admissible allocations, all of which

are evaluated end-to-end on the frozen targets. The rule does not see the outcomes, it uses only per-sub-task attributes, measured while the other three steps remain at their tool implementations, as assumed in §4.

To separate selection from evaluation, we use leave-one-target-out cross-fitting. For each target, attributes are estimated on the other 23, the rule selects an allocation, and that allocation is evaluated on the held-out target. It selects $TLTT$ on every fold and obtains loss 0.604, statistically indistinguishable from 0.594 for the best fixed allocation $TTTT$ (paired gap 0.010, 95% CI $[-0.103,\ +0.141]$). A per-target oracle obtains 0.474, leaving 0.130 $[+0.043,\ +0.235]$ that could only be recovered by an adaptive policy.

Replacing the assumed η_k with concentration bounds produces the same allocation, but the certificate remains loose. With 24 targets, its slack is 0.347 for most sub-tasks; only h_3, estimated from 6619 frame-level judgments, has a useful slack of 0.021. Reaching 0.05 throughout would require approximately $1{,}150$ independent units. The selected allocation recurs in 0.45 of the 80 tested weight settings (Appendix Figure 3). This 45% share describes sensitivity to the tested design priorities, not a probability of deployment robustness; the other settings select different allocations. Lemma 1 bounds sampling error under its independence assumptions, but does not cover changes in deployment distribution.

Sensitivity to deployment assumptions. For fixed weights, if deployment changes each attribute by at most $\epsilon_{e,k}$, $\epsilon_{t,k}$, $\epsilon_{c,k}$, and $\epsilon_{\phi,k}$ for either mechanism, its local loss changes by at most

$$\epsilon_{d,k} = w_{\mathrm{A}}\epsilon_{e,k} + w_{\mathrm{T}}\epsilon_{t,k}/T_0 + w_{\mathrm{C}}\epsilon_{c,k}/C_0 + w_{\mathrm{F}}\epsilon_{\phi,k}.$$

Consequently, a measured LLM–tool loss gap greater than $2\epsilon_{d,k}$ preserves its sign; smaller gaps warrant recalibration. The development gaps in Table 5 make parsing (0.143) and selection (0.727) more sensitive than retrieval (1.633) and visibility (7.632) under that operating point. These margins are diagnostic, since independent labels already revised the visibility estimate.

Flexibility changes with the request mix: a parser restricted to canonical tags loses less flexibility than one serving multilingual paraphrases, and a selector compiled for a closed set of criteria differs from one facing new constraints. Holding the other development attributes fixed, $\widehat{\Delta}_1 = -0.0133 - 0.65w_{\mathrm{F}}$ and $\widehat{\Delta}_4 = 0.84694 - 0.60w_{\mathrm{F}}$, so selection switches from tool to LLM at $w_{\mathrm{F}} \simeq 1.41$. If the deployed parser serves the same variants as the LLM, its flexibility penalty vanishes and its gap becomes -0.0133; if its error also drops from 0.15 to 0.02, its gap becomes $+0.1167$, favoring the tool. These are conditional calculations, not additional experiments. Hardware and network round trips change latency; token counts proxy cost here, while monetary deployment cost would include API rates, tool hosting, and amortized synthesis. Missing pose, camera calibration, terrain, and occlusion alter the visibility contract and its measured error. A deployment should therefore fix its request mix, metadata assumptions, resource accounting, and priorities before re-estimating attributes and applying the allocation rule.

6 Conclusion

This work supports a broader distinction between what an agent should infer and what it should compute. LLMs remain useful when a step depends on language, ambiguity, or changing user intent, but physical quantities fixed by camera geometry should not be repeatedly reconstructed through prompting. The experiments make this distinction more precise: providing the equations does not make the model reliably carry out the computation, while asking it to write and execute code can close the precision gap. The model can therefore be good at constructing a computational procedure without being equally reliable at executing that procedure inside its forward pass. Moreover, a criterion that depends on the request does not necessarily require an LLM at serving time: it may be learned or specified once, compiled into a deterministic selector, and then reused more cheaply. Our cross-fitted evaluation supports this design view, as the selected allocation performs similarly to the best fixed allocation, while poorly assigning the mechanisms produces substantially greater loss. The main conclusion is therefore not that tools are always preferable, but that agents should use LLMs for flexibility, explicit computation for physically determined quantities, and independent ground truth to decide where the boundary lies.

References

Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998.

Laming Chen, Guoxin Zhang, and Eric Zhou. Fast greedy MAP inference for determinantal point process to improve recommendation diversity. In *Advances in Neural Information Processing Systems*, volume 31, pages 5627–5638, 2018.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Yuval Eldar, Michael Lindenbaum, Moshe Porat, and Yehoshua Y. Zeevi. The farthest point strategy for progressive image sampling. *IEEE Transactions on Image Processing*, 6(9):1305–1315, 1997.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning*, 2023.

Teofilo F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306, 1985.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

Om Khangaonkar, Hadi J. Rad, and Hamed Pirsiavash. Multimodal language models cannot spot spatial inconsistencies. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026.

Miho Koda, Yu Zheng, Ruixian Ma, Mingyang Sun, Devesh Pansare, Fabio Duarte, and Paolo Santi. LocationReasoner: Evaluating LLMs on real-world site selection reasoning. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026.

Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286, 2012.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

Zhanpeng Luo, Ce Zhang, Silong Yong, Cunxi Dai, Qianwei Wang, Haoxi Ran, Guanya Shi, Katia Sycara, and Yaqi Xie. Explicit 3D spatial reasoning via program generation. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026.

Riddhiman Rana, Aryav Semwal, Yogesh Atluru, Shivank Garg, Cristian Meo, and Kevin Zhu. Orion: A fully deterministic and interpretable pipeline for video scene graph generation with explicit causal influence scoring. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Josephine Wang, Julien Coquet, and Jeffrey Huang. CivicEmbed: Feature-specific embeddings for efficient geographic reasoning and retrieval. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026a.

Suyuchen Wang, Tianyu Zhang, Ahmed Masry, Christopher Pal, Spandana Gella, Bang Liu, and Perouz Taslakian. Improving GUI grounding with explicit position-to-coordinate mapping. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Yuxi Xiao, Longfei Li, Shen Yan, Xinhang Liu, Sida Peng, Yunchao Wei, Xiaowei Zhou, and Bingyi Kang. SpatialTree: How spatial abilities branch out in MLLMs. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026. Best Paper Award.

Zelin Xu, Yupu Zhang, Saugat Adhikari, Saiful Islam, Tingsong Xiao, Zibo Liu, Shigang Chen, Da Yan, and Zhe Jiang. EarthSpatialBench: Benchmarking spatial reasoning capabilities of multimodal LLMs on Earth imagery. In *ICLR 2026 Workshop on Efficient Spatial Reasoning*, 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Dazhou Yu, Riyang Bao, Ruiyu Ning, Jinghong Peng, Gengchen Mai, and Liang Zhao. Spatial-RAG: Spatial retrieval augmented generation for real-world geospatial reasoning questions. *arXiv preprint arXiv:2502.18470*, 2025.

A Proof

Proof of Proposition 1.

For each k,

$$d_k(\widehat{a}_k) \le \widehat{d}_k(\widehat{a}_k) + \eta_k \le \widehat{d}_k(a_k^\star) + \eta_k \le d_k(a_k^\star) + 2\eta_k.$$

Summing over the sub-tasks and using $\underline{S}(a) = w_{\mathrm{A}} + w_{\mathrm{F}} - \sum_k d_k(a_k)$ proves (10). Furthermore, an accepted tool with $\widehat{d}_k(a_k) - \widehat{d}_k(\tau_k) > 2\eta_k$ satisfies

$$d_k(a_k) - d_k(\tau_k) \ge \widehat{d}_k(a_k) - \widehat{d}_k(\tau_k) - 2\eta_k > 0,$$

so the replacement strictly lowers the true local loss. □

Lemma 1 (Simultaneous mechanism-error certificate)**.** *Consider K sub-tasks, each with at most M candidate mechanisms. For each pair (k, m), let $\widehat{e}_k(m)$ be the mean of $n_k(m)$ independent Bernoulli error observations with expectation $e_k(m)$. Then, for any $\delta \in (0, 1)$, with probability at least $1 - \delta$,*

$$|e_k(m) - \widehat{e}_k(m)| \le \sqrt{\frac{\log(2M/\delta_k)}{2n_k(m)}}, \qquad \delta_k := \frac{\delta}{K}, \tag{12}$$

simultaneously for every sub-task k and candidate mechanism m.

Proof. For a fixed pair (k, m), Hoeffding's inequality gives

$$\Pr(|e_k(m) - \widehat{e}_k(m)| > \epsilon) \le 2\exp\left(-2n_k(m)\epsilon^2\right).$$

Setting the right-hand side to δ_k/M yields the bound in (12). A union bound over the M mechanisms gives failure probability at most δ_k for each sub-task, and a second union bound over the K sub-tasks gives total failure probability at most $\sum_{k=1}^{K} \delta_k = \delta$. □

B Site, Data and Metadata Gaps

All cases use one surveyed industrial site. The asset catalogue holds 109 tagged assets with geodetic positions and, for most, a surveyed height and ground extent. The image table holds 31,213 geo-located frames from drone and handheld capture, each carrying position, compass yaw, gimbal pitch, absolute and relative altitude, and — for part of the set — a 35 mm-equivalent focal length.

Table 3 reports how the two collections interleave, averaged over assets. Density saturates quickly: beyond 50 m every asset has the full catalogue as neighbours, and the frame count grows from $\sim$3.8k at 25 m to $\sim$15k at 500 m and then flattens. This is what makes proximity a weak signal for the retrieval task: a 500 m radius already contains half the corpus, so ranking by distance ranks within a pool that is not selective.

Table 3: Mean number of catalogued assets and geo-located frames within a given radius of an asset.

Radius (m)	Mean assets	Mean frames
25	82.9	3,848
50	108.0	8,479
100	108.0	9,501
250	108.0	11,839
500	108.0	15,085
1000	108.0	15,118

A metadata gap shapes the design. Specifically, camera orientation is not universally recorded. Yaw, pitch and relative altitude appear under several vendor-specific keys, sometimes several within one export, and a stored relative altitude of exactly zero means *missing* rather than ground level for an aerial frame. Frames whose pose cannot be reconstructed are classified as *pose unknown* by τ_3 and counted against coverage, since an unverifiable frame is not evidence that the asset is visible.

C Field-of-View Geometry and Back-Projection

Both τ_3 (Case B) and the region tool of Case A are directions of one projection, computed in a local east–north–up frame with the target at the origin and a ground plane $z = 0$. All geometric correctness claims are relative to this pinhole-camera model, the ground-plane approximation, and valid pose metadata; they do not certify visibility under unmodelled terrain relief or occlusion.

Optics. From a 35 mm-equivalent focal length f, the horizontal and vertical fields of view are

$$\theta_h = 2\arctan\left(\frac{36}{2f}\right), \qquad \theta_v = 2\arctan\left(\frac{24}{2f}\right),$$

against a 36×24 mm reference sensor. When f is unavailable we use the wide-lens defaults $\theta_h = 48.5^\circ$ and $\theta_v = 33.4^\circ$, which match the camera that produced most of the corpus.

Footprint (forward direction, τ_3). Let d, r, u be the camera forward, right and up vectors built from its compass yaw and gimbal pitch. Each image corner $(s_h, s_v) \in \{-1, 1\}^2$ defines the ray

$$q_{s_h,s_v} = d + s_h \tan(\theta_h/2)\, r + s_v \tan(\theta_v/2)\, u.$$

Intersecting the four rays with $z = 0$ — clamping range so that a shallow-pitch camera yields a bounded polygon rather than one reaching the horizon — gives a four-corner ground footprint, and the target is in-frame when its ground position lies inside that polygon by a ray-casting test. The result is three-valued: `in`, `out`, or `pose unknown` when yaw, pitch or height cannot be recovered.

Two worked cases. A nadir camera at 60 m with the default optics has footprint half-widths $60\tan(48.5^\circ/2) \approx 27$ m and $60\tan(33.4^\circ/2) \approx 18$ m. Placed 10 m east of the target its footprint spans $E \in [-17, 37]$, $N \in [-18, 18]$ and the target at the origin is inside; moved to 40 m east the footprint becomes $E \in [13, 67]$ and the target is outside. An oblique camera 40 m east with yaw 270° and pitch -35° projects to a trapezoid spanning approximately $E \in [-141, -7.4]$, $N \in [-82, 82]$; because the trapezoid stops at $E = -7.4$ m the target is again outside. Neither distance nor camera-to-target angular alignment predicts these outcomes on its own, which is what the prompt of Appendix D asks a model to infer.

Back-projection (inverse direction, Case A). Given the same oriented camera, the target's position is expressed in the camera frame and divided through by depth to obtain a point in the image plane, then scaled to x, y normalized to the full original frame and carried alongside `img_w`/`img_h` so a consumer can convert to pixels or remap onto a cropped derivative. Projecting the asset's surveyed ground extent and height, when the catalogue records them, yields a region rather than a point; where extent is unknown the tool returns the point only, because a box would imply knowledge of extent it does not have. The output is $\varnothing$ when the target does not project inside the frame, and it is deliberately not a default position: a marker at $(0, 0)$ renders a plausible crosshair in a corner and means nothing.

Redundancy metric. Viewpoint redundancy for a selected set S is the mean pairwise Gaussian similarity of camera positions,

$$\mathrm{sim}(S) = \frac{1}{|S|(|S|-1)} \sum_{i \neq j \in S} \exp\left(-\frac{\|x_i - x_j\|_2^2}{2\sigma_S^2}\right),$$

with σ_S set from the candidate-set scale, so lower is better. The same quantity, multiplied by a distance-decaying quality per candidate, forms the DPP kernel used by the baselines of Appendix I, which keeps the metric and the competing objective on a common footing.

D Prompts and Model Configuration

D.1 Case A: single-call visual grounding

The baseline of Section 5.1 is one call to `gemini-3.7-flash` with the frame attached, a JSON response schema of `{found: boolean, box_2d: integer[4]}`, and the instruction below. The explicit permission to answer `found=false` matters for the reading of Figure 1b: the failure is not that absence was inexpressible, but that it was not used.

```
User request: {prompt}

Locate only the requested visible image element. Return one tight bounding box
as [ymin, xmin, ymax, xmax], normalized from 0 to 1000. Include only the visible
part if the object is cropped or occluded. If it cannot be identified from this
image and request, set found=false and return an empty box.
```

Figure 1 is produced with the prompt `"Highlight the vessel AB230"` on two frames of the same asset; the returned boxes are rendered by scaling from the $[0, 1000]$ convention to pixels and compositing a translucent overlay, so the figure shows the model's raw output with no post-processing.

D.2 Case B: single-call retrieval

The single-call baseline receives the reference location, the neighbouring assets as occlusion proxies, and the candidate frames with raw camera geometry. Note that every quantity needed for the footprint test is supplied, including the camera-to-reference bearing, and that the instruction describes the decision procedure explicitly.

```
You are a geospatial vision expert. A user wants the {k} clearest
photographic views of a REFERENCE location on the ground: images whose
camera FIELD OF VIEW (FOV) actually contains that reference.

You are given the reference (lat, lon), nearby ASSETS that can occlude
the view, and a list of candidate IMAGES with RAW camera geometry only.
There is no precomputed "is it in frame" flag - you must infer it.

How to decide if the reference is inside a camera's field of view:
1. Angular alignment: compute the gap between camera_yaw_deg (the compass
direction the camera faces) and bearing_cam_to_ref_deg (the direction
from the camera to the reference). A small gap (roughly < 25 deg for a
typical lens) means the reference is near the optical axis and likely
in frame. A large gap means the camera looks away -> the reference is
NOT in frame, no matter how close it is.
2. Pitch + height + distance: camera_pitch_deg = -90 is straight down
(nadir). A near-nadir camera frames the ground around/below it, and its
ground footprint grows with camera_height_m; the reference must fall
inside that footprint given distance_to_ref_m. A shallow (near 0) pitch
frames the horizon and rarely covers a nearby ground point.
3. Occlusion: prefer images with few or no possible_occluders on the
line of sight.
Never reward closeness when the camera is not pointed at the reference.

Reference location:
{reference}
Assets (potential occluders):
{assets}
Candidate images (raw metadata):
{images}

Select the best {k} image ids whose field of view most clearly contains
the reference. Do your FOV reasoning internally; do NOT output it.
Return ONLY valid JSON, no prose, exactly:
{"selected": [<image id>, ...]}
Best first, with length <= {k}.
```

D.3 Case B: the agentic variant

The *LLM + FOV tool* row of Table 4 uses the identical prompt with one block inserted before the output instruction, and nothing else changed. It is the intermediate design discussed in Section 5.2: the geometry is available as a tool, but h_4 remains with the model.

```
You have a tool available:
 check_field_of_view(image_ids: [string]) -> {id: true|false}
It reports, for each requested image id, whether the reference lies inside
the projected footprint under the deployed camera model. Use it to VERIFY
your reasoning
before committing: reason first about which candidates are promising,
then confirm the promising ones (and, if useful, a few you are unsure
about) with the tool. Do not call it on every image blindly.
```

Automatic function calling is disabled so that inference time and tool time are metered separately. Each turn either returns a final selection or requests the tool; tool results are appended to the conversation and passed back in a new turn. If the turn limit is reached or no valid selection is produced, a final call with tools disabled and zero thinking budget forces a clean JSON response — which is why the reported token counts for this configuration include turns that contributed no selection.

D.4 Generation configuration

All Case B model configurations differ only in `thinking_budget`, set to 0, a low and a high value for the three single-call rows and to the high value for the agentic row. Temperature is fixed at 0.2 and the response MIME type constrains the output to JSON, so the accuracy differences in Table 4 are not decoding artefacts.

```
contents = [types.Part.from_text(text=prompt), *image_parts]

config = types.GenerateContentConfig(
    temperature=0.2,
    response_mime_type="application/json",
    max_output_tokens=thinking_budget + max_output_tokens,
    thinking_config=types.ThinkingConfig(thinking_budget=thinking_budget),
)

resp = client.models.generate_content(model=model, contents=contents, config=config)
```

D.5 The deployed agent's instruction

In the accepted allocation the model retains h_1 and the reporting of results, and its instruction is written to keep it there. Three clauses do the work; they are stated because they are the interface through which the tool's refusals survive to the caller.

```
1. NEVER invent an image. Every id you emit must have come from the tool's
   response. If you need more views than it returned, you do not have them.

2. REPORT SHORTFALLS HONESTLY. When the tool returns fewer views than
   requested, state the real number and the reason from ‘counts‘.
   - Correct: "Returning 3 views. Only 3 of 299 nearby images have the asset
     in frame; the other 218 point elsewhere and 78 have no usable camera
     heading. Widening the radius may find more."
   - WRONG: "Here are 10 views." (when 3 were returned)
   - WRONG: padding the list to 10 with ids you did not receive.

3. NEVER claim an image shows the asset unless the tool said so. Every view
   the tool returns satisfies the deployed geometry and metadata checks.
   You add nothing to that judgement and must not soften or embellish it.
```

The mechanisms for h_3 and h_4 are exposed to this agent as a single call. The decomposition is unchanged—τ_3 and τ_4 remain separate mechanisms with separate contracts and separate measured losses—but the agent is given no choice between them, because it has none to make: with h_2 allocated

to the calling agent, the two always run in sequence over whatever pool arrives. An earlier revision exposed them separately and additionally lent this agent the caller's database access so that it could drive its own retrieval loop. Both were withdrawn. Delegating h_2's mechanism to a sub-agent surrenders control over which queries are issued, not merely over which are valid; and once h_2 moves out, two steps that always run in one order are one step plus a protocol the model can get wrong. The candidate pool is passed as state rather than through the model, so neither model holds the rows.

E Case B: Development Query and Estimated Error

This appendix explains how the development measurements used in Case B were obtained. Table 4 reports the query that originally exposed the failure and motivated the pipeline. We keep it because it makes the failure mode concrete, not because one query can estimate how often that failure occurs. The corresponding population-level comparison is carried out on frozen targets in Section 5.5.

Table 4: Case B on the development query, using one shared candidate pool and $\kappa = 10$. **In-frame** is the fraction of returned frames whose projected footprint contains the target. **Redundancy** is the mean pairwise RBF similarity, so lower is better. The h_3, h_4 column gives the visibility and selection mechanisms; *skip* means that no visibility test is applied. Latency is end to end and internal tokens are model reasoning tokens.

Mechanism	h_3, h_4	Latency (ms/query)	In-frame (%)	Redund. ↓	Tok_{in}	Tok_{out}	Tok_{int}
Random κ	skip, tool	18	30	0.816	—	—	—
Top-κ nearest	skip, tool	34	50	1.000	—	—	—
LLM, no reasoning	LLM, LLM	2,089	0	0.948	19,360	348	0
LLM, low reasoning	LLM, LLM	3,905	0	0.823	19,360	379	489
LLM, high reasoning	LLM, LLM	20,857	0	0.823	19,360	377	4,095
LLM + FOV tool	tool, LLM	12,148	**100**	0.948	41,264	340	2,097
FOV filter + diversity	tool, tool	**120**	**100**	**0.782**	—	—	—

To populate Table 5, we measure each mechanism on the same candidate pool and exclude preprocessing shared by all alternatives. The reported times are medians over repeated runs, while token costs come directly from the API meter. These local measurements differ from the end-to-end latency in Table 4: the local visibility and selection kernels take only a few milliseconds, whereas the complete query also includes data preparation and conversion. This distinction prevents shared setup from being charged repeatedly when the local losses are combined.

Each error is defined at the output of its own sub-task. Parsing fails when h_1 returns the wrong tag or count; retrieval fails when h_2 omits frames required by its contract; visibility fails when h_3 gives the wrong in-frame verdict; and selection fails when h_4 returns an unknown identifier or an excessively redundant set. A declared refusal is not automatically an error: *pose unknown*, or a short answer accompanied by the corresponding exclusion counts, satisfies the contract. The values in Table 5 are therefore diagnostic estimates from the development query, not population error rates.

Flexibility $\widehat{\phi}_k$ is measured on a fixed list of task variants that each mechanism must serve without changing its code or interface. For example, the list for h_1 varies how the asset and requested count are expressed, while the list for h_3 includes conditions outside the ground-plane pinhole model. Unlike latency and token cost, these flexibility scores depend on the chosen variant list. They are consequently the least certain entries in Table 5; Proposition 1 represents this uncertainty through η_k, and Section 5.2 checks how the allocation changes with w_{F}.

All results in Tables 4, 5, and 9 are development results from one site. Mechanisms within a table share the same inputs and preprocessing, but Tables 4 and 9 use different targets and should not be compared row by row. These experiments explain how the design was formed; the frozen protocol in Section 5.5 is the test of whether it generalizes.

Table 5: Estimated attributes and local design loss for the two mechanisms available at each Case B sub-task. We use $w_{\mathrm{A}} = 1$, $w_{\mathrm{T}} = 0.3$, $w_{\mathrm{C}} = 0.2$, $w_{\mathrm{F}} = 0.2$, $T_0 = 1\,\mathrm{s}$, and $C_0 = 10^4$ tokens. Costs are reported in 10^3 tokens. Bold marks the lower-loss mechanism, and $\widehat{\Delta}_k$ is the gain from replacing the LLM with the tool.

	h_1 parse		h_2 retrieve		h_3 visibility		h_4 select	
	f_1	τ_1	f_2	τ_2	f_3	τ_3	f_4	τ_4
$\widehat{e}_k$	0.02	0.15	1.00	0.00	1.00	0.00	0.50	0.02
$\widehat{t}_k$ (s)	0.35	0.001	2.40	0.89	20.86	0.006	1.20	0.0002
$\widehat{c}_k$ (ktok)	0.6	0	12.0	0	23.8	0	0.35	0
$\widehat{\phi}_k$	0.95	0.30	0.90	0.60	1.00	0.50	1.00	0.40
$\widehat{d}_k$	**0.147**	0.290	1.980	**0.347**	7.734	**0.102**	0.867	**0.140**
$\widehat{\Delta}_k$	-0.143		$+1.633$		$+7.632$		$+0.727$	
allocation	retain f_1		accept τ_2		accept τ_3		accept τ_4	

F Case C: Criteria, Ground Truth and Pools

Case C keeps parsing, retrieval, and visibility fixed and changes only the final selection step h_4. The request now includes a criterion c, such as choosing thermal, overhead, close, or geographically distributed views. We compare three ways of handling it: a fixed selector designed for one objective, an LLM that interprets c at serving time, and τ_4^{synth}, for which an LLM writes a criterion-specific scoring function once and that function is reused thereafter. This experiment asks whether request-time language requires a model in the serving path, or whether the relevant criterion space can be compiled in advance.

Table 6: Case C comparison at h_4 over eleven request-time criteria and three assets, with $\kappa = 5$. Precision is the median over assets, and *serves* counts the criteria for which precision reaches 0.8. A mechanism is admissible only if its interface can receive the requested criterion. Synthesis tokens for τ_4^{synth} are paid once, offline.

Mechanism at h_4	Admissible	Precision	Serves	Latency	Tokens	$\widehat{d}_4$
τ_4 farthest point	no	0.33	1/11	0.2 ms	—	0.85
τ_4' top-κ nearest	no	0.20	0/11	0.02 ms	—	1.00
f_4 LLM selection	yes	0.72	7/11	2.6 s	4.6k	1.23
τ_4^{synth} compiled per criterion	yes	**0.818**	**9**/11	0.025 s	—	**0.226**

The eleven criteria and their individual scores appear in Table 7. Each answer can be computed directly from candidate metadata, independently of the mechanisms being compared. For a *rank* criterion, the reference answer is the true top-κ under the stated ordering; for a *filter* criterion, it is the set of candidates satisfying the stated predicate. The only exception is *spread*, whose reference is the deployed diversity selector. We retain this criterion because it is the objective for which that fixed selector was designed; removing it after seeing the results would bias the comparison against the tool.

Every mechanism receives the same input pool. We first run h_2 and h_3 exactly as in Case B: retrieve up to 400 frames within $500\,\mathrm{m}$, then retain only those for which τ_3 can establish visibility from a complete pose. The selector receives short candidate references containing the relevant metadata—distance, pitch, yaw, bearing, and modality—rather than long UUIDs. This keeps the experiment focused on interpreting the requested criterion instead of copying identifiers.

Two criteria remain difficult for every tested mechanism: *farthest* and *from the side*. The limitation is partly in the data rather than the selector. Some pools contain views separated by only a few metres, and most available images were captured from above, so the desired distinction is weak or absent. We report these criteria because they were fixed before evaluation; removing them would overstate the fraction of requests that any mechanism can serve.

Case C also provides a direct check on the flexibility score used for h_1. A separate suite contains 38 requests from six families: tag expressions, count phrasing, coordinates, coordinates written in words,

Table 7: Precision@κ for each request-time criterion, with $\kappa = 5$. Values are medians over three assets with 61, 54, and 28 in-FOV candidates.

Criterion ("κ views of $\rho \ldots$")	Kind	τ_4	τ_4'	f_4
closest to the asset	rank	0.20	0.20	1.00
farthest from it	rank	0.20	0.00	0.40
looking straight down	rank	0.60	0.00	1.00
low, shallow angle	rank	0.00	0.00	0.80
from the north side	rank	0.20	0.00	0.60
thermal images	filter	0.40	0.40	1.00
colour, not thermal	filter	0.60	0.60	1.00
spread out around the asset	tool	1.00	0.20	0.30
from the side rather than above	filter	0.00	0.00	0.00
close-up colour, for a corrosion check	filter	0.40	0.40	1.00
not overhead and not thermal	filter	0.00	0.40	0.80

relative references, and non-English requests. The pattern extractor serves only the narrow syntax for which it was programmed, whereas f_1 handles most families. This is why the Case B allocation retains the LLM for parsing even though a fixed parser is cheaper.

G Case D: Contract, Suite and Protocol

Case D asks whether an LLM can construct the visibility tool from its contract. The purpose is not to test unrestricted program synthesis, but to separate two possible sources of failure: the model may fail to implement a stated requirement, or the requirement may be missing from the specification. In both arms, the maker receives the interface below, three worked examples, and a prose description of the acceptance suite. It never sees the deployed implementation or the test code. The *core* arm receives the same material with the field-conditions block removed.

```
def in_field_of_view(row: dict, target_lat: float, target_lon: float) -> bool|None

Three-valued: True inside the footprint, False outside, None when the pose
cannot be reconstructed. Yaw is a compass bearing; pitch is negative downward.
Pose keys vary by vendor and are listed in priority order.

FIELD CONDITIONS this corpus imposes (withheld in the core arm):
 - Yaw AND pitch are both required. If either is missing the pose is unknown;
   assuming nadir reports an oblique frame that does frame the target as
   though it does not.
 - A relative altitude of exactly 0.0 means MISSING, not ground level. A row
   carries no ground elevation, so use a default camera height of 60 m.
 - Clamp the ground range to 500 m so a shallow pitch cannot reach the horizon.
 - With no focal length use 48.5 deg horizontal, 33.4 deg vertical.
```

The acceptance suite is fixed before any code is generated. Its eight *core* tests cover ordinary geometric cases: cameras facing toward, away from, and sideways to the target; two additional bearings; distant and overhead nadir views; and a row with no camera position. Five *field* tests encode conventions specific to this corpus: missing yaw, missing pitch, alternative vendor keys, zero relative altitude used as a missing value, and a shallow-pitch view whose footprint must be range-limited. The deployed tool passes all thirteen tests. Generated programs are run against the gate described to their arm. When a program fails or does not compile, the maker receives the failed assertions and may repair it up to three times. Pass@n is the fraction of synthesis attempts that satisfy the complete gate within n generated versions.

Passing the synthetic gate checks the written contract, but not transfer to real metadata. We therefore add the controlled test reported in Table 8: 300 real frames from three assets, balanced across the deployed tool's three verdict classes. Balancing is necessary because a tool that always returns `False` would otherwise appear accurate on this corpus. We compute agreement within each class and then average the classes, with a separate score for frames governed by a field condition. This diagnostic measures whether the specification is sufficient to reproduce the deployed behavior; it is not an independent ground-truth evaluation. The larger frozen experiment in the main paper provides that comparison using the oracle and pixel labels.

Table 8: Case D over ten synthesis attempts per arm. Pass@n measures success on the gate described to that arm. *All 13* evaluates every generated tool on the complete suite; *Real agr.* and *Field subset* measure transfer to the controlled real-frame set.

Specification	pass@1	pass@3	all 13	Real agr.	Field subset	η_3
with field clauses	1.00	1.00	1.00	0.990	0.964	0.01
without them	0.90	1.00	0.00	0.923	0.625	0.60

Table 8 separates compliance with a gate from adequacy of that gate. Both arms can produce code that passes the tests they are shown, but only the full specification prepares the maker for the withheld field cases. The missing assumptions cannot be recovered reliably from generic examples alone. This is the main lesson of Case D: synthesis quality depends on the completeness of the contract and on tests that exercise its deployment conditions.

The experiment has three limits. The alias test remains synthetic because every real frame available here uses the `PreciseCamera*` keys. The study also uses one model and one site, with only ten attempts per arm. The variation across core attempts is therefore reported rather than treated as a precise estimate of η_3.

H Frozen Held-Out Protocol

The frozen set is chosen without inspecting the outcomes. Starting from the 109 catalogued assets, we remove the four development targets (`T100-AB101`, `T100-AB105`, `T100-AA105`, and `T110-AK111`) and assets with fewer than 200 nearby frames. We preserve the proportions of the five asset categories and order candidates within each category by the SHA-256 hash of their tag. This produces 24 targets by a deterministic rule that cannot be tuned by trying different random seeds. For each target, the pool contains the nearest 400 frames within 500 m. All evaluated mechanisms receive the same frame bytes.

The oracle is implemented independently of the deployed FOV tool so that it does not reproduce the same code or parsing errors. For each frame, it reads the camera position, yaw, pitch, roll, and 35 mm-equivalent focal length from EXIF. It expresses the target offset in a local ENU frame, rotates that offset into camera coordinates, and projects it onto the image using $f_{\mathrm{px}} = f_{35} W/36$. A target is labelled *visible* only when it lies in front of the camera and its projected pixel (u, v) falls inside $[0, W] \times [0, H]$.

The oracle returns *uncertain* when the available metadata cannot support a stable decision: orientation is missing, the range exceeds 250 m, or the projection falls within 3% of an image edge. At long range the asset may occupy only a few pixels, while near the edge small lens and GNSS errors can reverse the verdict. The oracle imports no module from the deployed geometry, and this separation is checked at import time. The only shared quantity is ground elevation, estimated for each pool as the median difference between absolute and relative altitude because the frames provide no independent datum.

Independence does not make the oracle automatically correct. We therefore compare the oracle and deployed tool on pixels whenever possible. Frames are sampled by their joint pair of verdicts, with disagreements deliberately over-sampled and later reweighted to the recorded sampling design. The automatic judge is `gemini-3.7-flash` at temperature 0.4. It makes three independent decisions per frame, combined by majority vote, with ties mapped to *unsure*. The judge receives only the following two-image prompt—no camera pose, coordinates, or geometric verdict:

```
IMAGE 1 is a REFERENCE. The equipment of interest is at its centre.
IMAGE 2 is a CANDIDATE frame.

Question: is the specific piece of equipment shown at the centre of IMAGE 1
also present somewhere inside IMAGE 2?

Judge only from what you can see. You have no camera positions, no headings
and no map. Similar-looking equipment elsewhere on the site does NOT count --
decide whether this frame plausibly covers the same physical spot, using the
surrounding layout, pipe runs, roads and tank arrangement as context.

Answer with JSON only:
{"answer": "yes" | "no" | "unsure", "confidence": 0.0-1.0, "why": "..."}

Use "unsure" when the candidate is too oblique, too distant, too dark or too
blurred to tell. "unsure" is a good answer; guessing is not.
```

The reference is selected without consulting either geometric verdict: it is the nearest fetchable frame in which the target is most central. This prevents the mechanism under evaluation from choosing the evidence used to judge itself. Distance is computed directly from the two recorded positions and is not taken from either visibility implementation.

All 111 disagreements, together with a random sample of agreements, are also labelled by a human author, giving 228 human-labelled pairs. The pairs are shuffled, and the annotator sees neither geometric verdict nor the automatic label. Human labels replace automatic labels wherever both are available. When both the human and automatic judge give a yes/no answer, they agree on 0.974. Their lower three-class Cohen's κ of 0.42 mainly reflects different choices about when to abstain, especially for thermal-versus-visual and aerial-versus-ground comparisons. The corresponding oracle and tool scores are reported in the main paper.

This validation uses one human annotator rather than a panel, so it cannot measure disagreement between people or bound annotator bias. The released labelling pack and merge tools allow additional

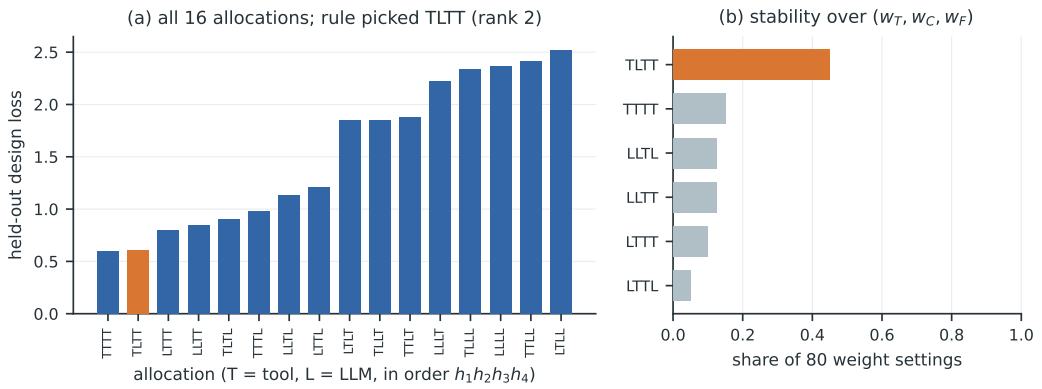


Figure 3: **(a)** All 16 admissible allocations by measured held-out design loss; orange is the one the rule selects from per-sub-task attributes alone. **(b)** Share of 80 weight settings on which the rule returns each allocation.

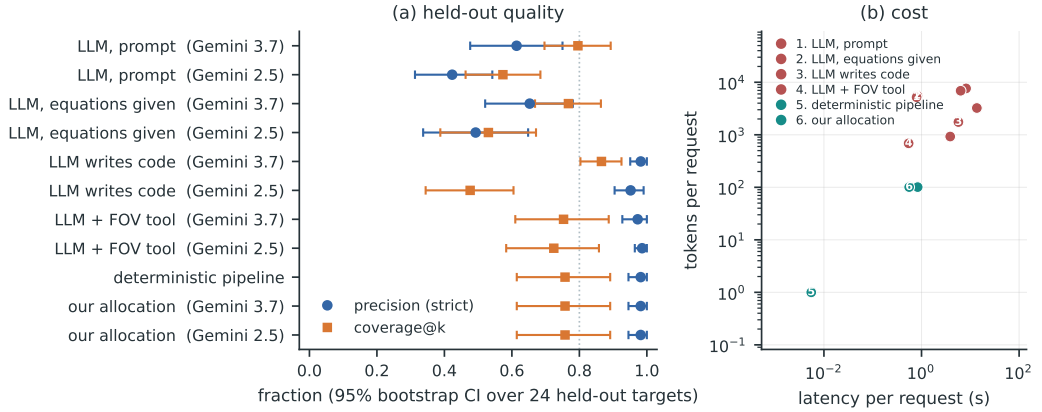


Figure 4: Frozen held-out evaluation over 24 assets disjoint from every development target. **(a)** Strict precision and coverage@κ with 95% bootstrap intervals over targets. **(b)** Latency against tokens, both logarithmic; teal marks the mechanisms with no model in the serving path.

annotations to be added without repeating the upstream experiment. This is the most direct way to strengthen the pixel reference in future work.

Pixel adjudication is also limited by image availability. The corpus serves 300×300 centre-cropped thumbnails from 4000×3000 originals. If a target projects inside the full sensor but outside the thumbnail crop, its absence from the available image cannot establish that it was absent from the original frame; these cases are excluded and counted. A target projected outside the full sensor remains adjudicable. Fetchable images are available for approximately 26% of frames, with similar rates across the oracle–tool verdict cells.

Table 2 in the main paper reports Gemini 3.7. We repeat the complete benchmark with Gemini 2.5 and obtain the same ordering of mechanisms. The important difference is the code-writing arm: its precision remains high, but its coverage falls from $0.865\ [0.803,\ 0.925]$ to $0.476\ [0.344,\ 0.605]$. The weaker model therefore produces conservative code that is usually correct when it returns a frame but misses many valid candidates. This variation is one reason to prefer a validated tool over generating new code for each request.

We also attempted to include an open-weight model family, but the available Gemma models could not complete the sweep under the same protocol. On a 120-row pool, each call required 87–115 s and exhausted the output allowance on hidden reasoning without returning an answer; increasing the allowance led to timeouts. Repeating this behavior over the required 360 calls would not produce a

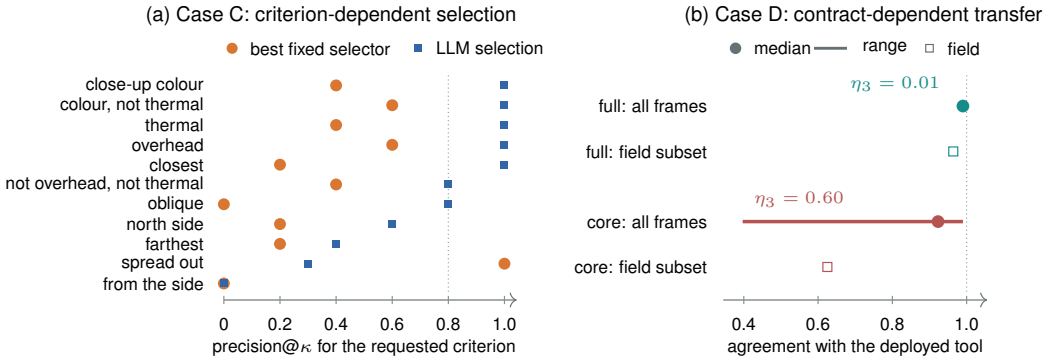


Figure 5: **(a) Case C.** Precision@κ at h_4 for each request-time criterion, comparing the best fixed selector with the LLM (Appendix F); the dotted line is the service threshold. **(b) Case D.** Transfer to 300 real frames. Filled circles show the median across ten synthesis attempts, horizontal segments show the range, and open squares show agreement on the 56 frames governed by field conditions.

meaningful benchmark. The evidence therefore supports robustness across the two evaluated models, not across model families.

I Extended Selection Sweep and Retrieval Scaling

This appendix checks whether the Case B failure can be repaired by changing only the final selector. We apply six standard selection methods to the same proximity-ranked pool before visibility filtering, and include the LLM selector and the accepted pipeline as references. Table 9 shows that selecting a diverse set does not ensure that the selected frames contain the asset: diversity and visibility are different objectives. The comparison also isolates the role of ordering. Farthest-point selection is evaluated once on the raw pool and once after τ_3 removes frames that do not contain the target. The selector is unchanged; only the input to it differs. This is why visibility must be established before optimizing viewpoint diversity.

Table 9: Selection algorithms over a proximity-ranked candidate pool, $\kappa = 10$. Latency is per selection decision; redundancy is mean pairwise similarity, lower being better. The last two rows are references rather than members of the sweep.

Selector	Latency (ms)	Redundancy ↓	In-frame (%)
Random κ	0.034 ± 0.042	0.816	30
Top-κ nearest	0.018 ± 0.012	1.000	0
Farthest point	0.165 ± 0.022	0.823	50
Greedy MMR	0.186 ± 0.183	0.871	20
k-DPP	0.850 ± 0.096	0.829	40
Greedy DPP-MAP	0.120 ± 0.064	0.827	60
LLM selector[‡]	568.4	0.948	100
FOV filter + farthest point	0.202 ± 0.072	**0.782**	**100**

[‡]The model selector performs differently here and in Table 4, which uses another target. Both runs are reported. Their difference illustrates the instance-to-instance variation hidden by a single estimate $\widehat{e}_3(f_3)$ and represented by η_3 in Proposition 1.

The implementation of τ_2 retrieves the candidate pool in one bulk query because database round trips dominate the runtime. One radius query returns 1,000 frames in 0.890 s. Querying candidates one at a time instead creates the standard $N{+}1$ pattern and requires 31.1 s for $\kappa = 10$, mostly in network round trips; retrieving a smaller nearest-neighbor pool first still takes 12.6 s. After the bulk query, all remaining steps run in memory. Their cost is small by comparison, so the accepted pipeline's total latency is determined mainly by retrieval rather than by the visibility filter or diversity selector.